

ECOM EXPERTS

Control de Horas - Servicio Técnico

Manual de usuario e instalación

ecom_controlhoras 1.2.0 · 04/09/2026

Contenido

1. Qué hace y para quién

2. Compatibilidad

3. Instalación

Qué crea la primera vez que se abre

Desinstalar

4. Configuración

5. Cómo se usa, día a día

Dar de alta un cliente con sus horas

Anotar un servicio (lo de cada día)

Leer el saldo

Consultar, corregir y filtrar

Los informes

Copia de seguridad

Atajos

6. Qué ve el cliente

7. Preguntas frecuentes y problemas

8. Seguridad

Lo que se ha comprobado y estaba bien

Lo que se ha arreglado en esta revisión

Lo que queda pendiente, y por qué

9. Ficha técnica

Aplicación de escritorio para Windows que lleva la cuenta de las horas de servicio técnico que consume cada cliente. Versión 1.2.0.

Sobre las capturas de pantalla de este manual: no las hay. Este proyecto **no es un módulo de PrestaShop ni una aplicación web**: es un programa de escritorio (.NET 8 + WPF) que se ejecuta como una ventana de Windows, así que no se puede levantar con un servidor ni fotografiar con un navegador. Todo lo que describe este manual está comprobado leyendo el código fuente y el ejecutable compilado, no supuesto.

1. Qué hace y para quién

Está pensado para una empresa de servicio técnico o de mantenimiento informático que vende **bolsas de horas**: el cliente contrata veinte horas, y a partir de ahí cada intervención —una llamada, una visita, un trabajo en remoto— le va restando tiempo de ese saldo. El problema de siempre es que ese descuento se lleva en una hoja de cálculo que nadie actualiza, y cuando el cliente pregunta «¿cuántas horas me quedan?» hay que ponerse a sumar celdas. Esta aplicación existe para que esa pregunta se conteste en un segundo y con un número en el que se pueda confiar.

El uso diario se reduce a una sola barra que está siempre a la vista: se escribe la descripción del trabajo, se escribe la duración y se pulsa **Intro**. El servicio queda anotado, el saldo del cliente se recalcula al instante y el cursor vuelve al principio para anotar el siguiente. Está medido para que anotar un servicio cueste menos de cinco segundos, porque si cuesta más, no se anota.

La duración se puede escribir como salga: **90**, **1h30**, **1:30**, **1,5** o **45m** son todas válidas, y además hay botones rápidos de 15 minutos, media hora, una hora, dos, cuatro y ocho. Todos los cálculos se hacen internamente **en minutos enteros** y se muestran como **2h 10m**, nunca como horas decimales: **2,17 h** no se lee de un vistazo y, al restar una y otra vez, arrastra error de redondeo.

Cuando hace falta enseñarle el consumo al cliente, hay dos informes imprimibles: el de un cliente concreto (su saldo, sus paquetes de horas contratadas, en qué tipo de servicio se le ha ido el tiempo y el detalle de cada intervención) y el general, que pone en una sola página a **todos** los clientes de un periodo, avisa de cuáles tienen las horas agotadas y reparte las horas por técnico. Ambos se abren en el navegador y con **Ctrl+P** se guardan como PDF. Además hay exportación a CSV para quien prefiera trabajarlo en Excel.

Importante: la aplicación no maneja dinero. No hay tarifa por hora, no hay importes y no hay facturas. Controla horas, y solo horas.

2. Compatibilidad

Sistema operativo	Windows de 64 bits (Windows 10 y 11 comprobados). Compilado para <code>win-x64</code> .
.NET	No hace falta instalarlo. El ejecutable es <i>self-contained</i> : lleva dentro el motor de .NET 8.
Permisos	Ninguno especial. No requiere administrador ni instalador.
Base de datos	SQLite, un único fichero. No hay servidor que montar.
Dependencias externas	Una sola: <code>Microsoft.Data.Sqlite</code> 8.0.10, ya incluida en el .exe.
Idioma	Castellano. La interfaz fuerza la cultura <code>es-ES</code> (fechas <code>dd/MM/yyyy</code> , decimales con coma).
Red / internet	No usa ninguna. No hace peticiones, no envía datos y no comprueba actualizaciones. Funciona sin conexión.
Multiusuario	Está pensado para un usuario a la vez . Ver el apartado 7 sobre el uso en carpeta compartida.
Tamaño	Un fichero de unos 64 MB.

Para **compilarlo** (solo si hay que tocar el código) hace falta el SDK de .NET 8; el guion `install-requisitos.ps1` lo instala si no está.

3. Instalación

No hay instalador y no hay nada que registrar en el sistema.

1. Copiar `dist\ControlHoras.exe` donde interese: el Escritorio, `C:\Aplicaciones\`, un pendrive, una carpeta de red...
2. Doble clic.

La primera vez Windows puede mostrar el aviso «**Windows protegió su PC**». Es SmartScreen avisando de que el ejecutable no está firmado digitalmente, no de que tenga nada raro. Se pulsa **Más información** → **Ejecutar de todas formas** y no vuelve a preguntar.

Qué crea la primera vez que se abre

Al arrancar crea la base de datos si no existe, con cuatro tablas (`clientes`, `bolsas`, `servicios` y `ajustes`) y sus índices. La creación es *idempotente*: usa `CREATE TABLE IF NOT EXISTS`, así que abrir la aplicación mil veces no destruye nada.

Dónde se guarda la base de datos (la aplicación decide en este orden, y muestra siempre la ruta activa abajo a la derecha de la ventana):

1. Si existe un fichero `controlhoras.db` **junto al** `ControlHoras.exe` → se usa ese.
2. Si existe una carpeta `datos` junto al `.exe` → se usa `datos\controlhoras.db`.
3. Si no → `C:\Users\<usuario>\AppData\Roaming\ControlHoras\controlhoras.db`.

Los casos 1 y 2 son el **modo portable**: basta con crear una carpeta vacía llamada `datos` al lado del ejecutable para que todo viaje junto en un pendrive o quede en una unidad de red.

Desinstalar

Se borra el `.exe`. No hay entradas en el registro, ni servicios, ni tareas programadas, ni ficheros repartidos por el sistema. Si además se quiere borrar el histórico, hay que eliminar a mano la carpeta `%APPDATA%\ControlHoras` (o la carpeta `datos` en modo portable). **Borrar el ejecutable no borra los datos**, que es justo lo que interesa al cambiar de versión: se sustituye el `.exe` y el histórico sigue ahí.

4. Configuración

Esta aplicación no tiene pantalla de configuración, y es a propósito: no hay nada que configurar antes de empezar a usarla. Solo hay tres decisiones, y las tres se toman desde la propia ventana principal.

El aspecto. El botón de la esquina superior derecha despliega cuatro temas visuales: *Clásico*, *Windows 11*, *Mac* y *Crystal*. El cambio se aplica en caliente, sin reiniciar, y queda guardado en la propia base de datos (tabla `ajustes`, clave `tema`), así que la próxima vez abre con el que se dejó. Por defecto es Windows 11. Es puramente estético: no cambia ningún cálculo.

Dónde viven los datos. Es la única decisión con consecuencias, y se toma **creando o no creando una carpeta**. Si se deja tal cual, cada usuario de Windows tiene su propio histórico en su perfil, protegido por su cuenta. Si se crea una carpeta `datos` junto al `.exe`, el histórico pasa a esa carpeta: es lo que hay que hacer para llevarlo en un pendrive o para compartirlo en un servidor. Antes de poner la base de datos en una carpeta compartida, conviene leer el apartado 7, porque tiene letra pequeña.

El técnico por defecto. No se configura: se aprende solo. El último nombre de técnico que se escribe al anotar un servicio se guarda (tabla `ajustes`, clave `tecnico_defecto`) y aparece ya puesto la próxima vez. El desplegable ofrece además todos los nombres que se hayan usado alguna vez.

5. Cómo se usa, día a día

Dar de alta un cliente con sus horas

Botón `+ Nuevo cliente` o `Ctrl+N`. Todo se rellena en la misma ventana, incluidas las horas contratadas: no hay un segundo paso.

Campo	Qué hace
Nombre / Empresa	Lo único obligatorio. Si se deja vacío, avisa y no guarda.
CIF/NIF, Persona de contacto, Email, Teléfono	Informativos. Salen en la cabecera del informe del cliente y sirven para buscar.
Horas	Las horas contratadas. Aquí un número suelto son horas (10 = 10 horas), a diferencia de la barra de servicios. También acepta 7,5 o 10h30 . Vacío = todavía no ha contratado nada.
Fecha	Cuándo contrató esas horas. Hoy por defecto.
Descripción	El concepto del paquete: «Bono mantenimiento 2026». Si se deja vacío, se guarda como «Horas contratadas».
Cliente activo	Al desmarcarlo el cliente desaparece de la lista sin perder su histórico. Vuelve a verse con la casilla Mostrar clientes inactivos .
Notas	Texto libre.

Si el cliente amplía horas más adelante, **no se edita el paquete anterior**: se añade otro con el botón **+** **Horas contratadas** de la cabecera. El saldo suma todos los paquetes, y en la pestaña **Horas contratadas** se ven, se editan y se borran uno a uno.

Cuando un cliente tiene **más de un paquete**, los campos de horas del formulario de cliente se bloquean y muestran el total: editarlos ahí sería ambiguo (¿cuál de los tres se está tocando?). Se gestionan desde la pestaña. Y si se **vacía** el campo de horas de un cliente que tenía un único paquete, la aplicación avisa de que va a eliminar esas horas y pide confirmación antes de hacerlo.

Anotar un servicio (lo de cada día)

Es la barra **ANOTAR SERVICIO**, siempre visible en cuanto hay un cliente seleccionado:

1. Descripción del trabajo.
2. Duración —escrita o con un botón rápido: **15m**, **30m**, **45m**, **1h**, **1h30**, **2h**, **4h**, **8h**.
3. **Intro**.

La fecha, el tipo y el técnico **se conservan** entre altas, así que en una tanda de anotaciones solo hay que teclear descripción y duración. **Ctrl+S** lleva el cursor a la descripción desde cualquier sitio.

Los formatos de duración admitidos, con la regla que los separa:

Se escribe	Significa	Por qué
90	90 minutos	Número entero suelto = minutos.
1,5 · 1.5	90 minutos	Número con decimales = horas.
1h30 · 1h 30m · 1:30	90 minutos	Formato explícito.
45m · 45 · 0,75 · 0:45	45 minutos	

Si lo escrito no encaja en ninguno, no guarda nada y avisa en la barra de estado con los ejemplos válidos.

La casilla «Descuenta» decide si ese tiempo resta del saldo. Marcada (lo normal), resta. Desmarcada, el servicio queda registrado en el histórico y sale en los informes, pero **no consume horas**: es lo que hay que usar para cortesías, trabajos en garantía o tiempo interno que se quiere dejar por escrito sin cobrárselo al cliente.

Leer el saldo

Cada cliente de la lista lateral lleva una barra de color:

Color	Estado	Cuándo
Verde	Con saldo	Menos del 80 % de las horas consumido.
Naranja	Queda poco	80 % o más consumido.
Rojo	Agotado	Ya no quedan horas (el saldo puede ser negativo).
Gris	Sin horas	El cliente no tiene ningún paquete contratado; solo se acumulan las horas usadas.

Al seleccionar un cliente, la cabecera muestra **Contratadas, Consumidas, Restantes y Servicios**, más el porcentaje consumido. Los tres tiempos van siempre en horas y minutos.

El saldo se calcula así: **contratadas** es la suma de todos los paquetes del cliente; **consumidas** es la suma de los minutos de los servicios marcados como «Descuenta»; **restantes** es la resta, y **puede ser negativa** si se ha trabajado de más (se muestra en rojo con el signo delante: **-1h 30m**). Los contadores se calculan sobre **todos** los servicios del cliente, no sobre los que deja ver el filtro de fechas: cambiar el filtro cambia la tabla, nunca el saldo.

Cuando un servicio deja el saldo a cero o en negativo, salta un aviso en el momento de anotarlo, con las tres cifras.

Consultar, corregir y filtrar

En la pestaña **Servicios**: doble clic para editar, clic derecho para *Editar / Duplicar / Eliminar*, y **Supr** para eliminar el seleccionado (siempre con confirmación). **Duplicar** copia el servicio a la barra de alta con la fecha de hoy, que es lo cómodo para trabajos repetitivos del tipo «revisión mensual».

Hay filtros **Desde** y **Hasta**, un botón **Este mes**, un cuadro de búsqueda que mira en la descripción, el técnico y el tipo, y un botón **Limpiar**. Al pie se ve siempre cuántos servicios hay en el filtro, cuánto tiempo suman y cuánto de ese tiempo descuenta.

Los informes

Informe del cliente (botón de la cabecera). Genera una página imprimible con el saldo y su barra de consumo, la tabla de paquetes de horas contratadas, el reparto del tiempo por tipo de servicio y el detalle de las intervenciones con sus totales. **Respetar los filtros de fecha activos**, así que sirve tanto para el resumen del mes como para el histórico completo.

Informe general de uso (botón inferior izquierdo). Primero pregunta el periodo —*Este mes, Mes pasado, Este año, Todo*, o fechas a mano—, si incluir a los clientes inactivos y si incluir el detalle servicio a servicio. Produce las cifras globales, un aviso con cuántos clientes están agotados o por encima del 80 %, la tabla resumen de todos los clientes ordenada por horas consumidas en el periodo, el uso por tipo de servicio, las horas por técnico y, opcionalmente, el detalle agrupado por cliente.

Los dos se guardan donde se indique, en formato `.html`, y se abren solos en el navegador. Desde ahí, `Ctrl+P` → **Guardar como PDF**. La hoja de estilos ya está preparada para imprimir: quita el fondo, los márgenes de pantalla y evita cortar un cliente entre dos páginas.

Exportar CSV vuelca los servicios del filtro actual con las columnas

`Fecha;Tipo;Descripcion;Tecnico;Duracion;Minutos;Horas;Descuenta`, y debajo las líneas de totales (servicios, tiempo total, lo que descuenta, horas contratadas y restantes). Va separado por punto y coma y con marca UTF-8, que es lo que Excel en castellano espera: se abre con doble clic y con los acentos bien. La columna `Minutos` es el número entero exacto y la columna `Horas` la versión decimal (`2,17`), que es la cómoda para sumar en Excel.

Las cabeceras del CSV van **sin tildes a propósito**: son nombres de columna que otros procesos pueden estar leyendo, no texto para una persona. Cambiarlas rompería cualquier hoja de cálculo o macro que ya apunte a ellas.

Copia de seguridad

Botón **Copia de seguridad**, abajo a la derecha. Crea una copia fechada en una subcarpeta `backups\` junto a la base de datos, con nombre `controlhoras_AAAAMMDD_HHMMSS.db`, y enseña la ruta exacta. Usa el mecanismo propio de SQLite, así que la copia es coherente aunque la aplicación esté abierta.

La copia manual también vale: el fichero `controlhoras.db` **es** todo el histórico. Copiarlo a otro equipo, a la misma ruta, es toda la migración que hay.

Atajos

Atajo	Acción
Ctrl + N	Nuevo cliente
Ctrl + F	Ir al buscador de clientes
Ctrl + S	Ir a la descripción del nuevo servicio
Intro	Guardar el servicio
Supr	Eliminar el servicio o el paquete de horas seleccionado
Doble clic	Editar

6. Qué ve el cliente

El cliente final **no entra en la aplicación**: no hay portal, ni web, ni acceso externo de ninguna clase. Lo único que llega a sus manos es lo que se le mande: el informe HTML (o su PDF) y, si se quiere, el CSV.

En el informe de cliente ve su nombre y sus datos de contacto, el saldo con la barra de consumo, sus paquetes de horas con fecha y concepto, el reparto por tipo de servicio y el detalle de cada intervención con la descripción, el técnico, la duración y si descontaba o no. Es decir: ve lo que se le anotó, tal y como se le anotó. Conviene tenerlo presente al escribir las descripciones y al elegir la casilla «Descuenta», porque el «No» en esa columna es exactamente lo que hace visible una cortesía.

El informe general de uso no se le manda a un cliente: contiene a todos los demás.

7. Preguntas frecuentes y problemas

«**Windows protegió su PC**» al abrirlo. El ejecutable no está firmado digitalmente. *Más información → Ejecutar de todas formas*. Solo la primera vez. Firmarlo requiere un certificado de firma de código, que se compra aparte.

No veo a un cliente en la lista. Casi siempre está marcado como inactivo. Marcar **Mostrar clientes inactivos** abajo a la izquierda. Si tampoco aparece, comprobar que el cuadro de búsqueda está vacío.

«**Duración no válida**». El texto escrito no encaja en ningún formato. Valen 90, 1h30, 1:30, 1,5 y 45m. Ojo a la regla que más despista: 2 son **dos minutos** en la barra de servicios, pero **dos horas** en el campo de horas contratadas del cliente.

No me deja editar las horas desde la ficha del cliente. Ese cliente tiene varios paquetes de horas. Se editan en la pestaña **Horas contratadas**.

El saldo sale en rojo y con un número negativo. Es correcto: se ha trabajado más de lo contratado. Se arregla añadiendo un paquete nuevo con `+ Horas contratadas`.

Puse una fecha de caducidad a un paquete y sigue contando. Cierto, y conviene saberlo: la fecha de caducidad **se guarda y se muestra** en la tabla y en el informe, pero **el saldo no excluye los paquetes caducados**. Hoy es un dato informativo, no una regla de cálculo. Está anotado como pendiente en el apartado 8.

Quiero pasar los datos a otro ordenador. Copiar `controlhoras.db` (la ruta sale abajo a la derecha) a la misma carpeta del otro equipo. O pasar a modo portable y llevarse la carpeta entera.

¿Puede usarlo mi compañero a la vez desde la carpeta de red? No se recomienda. La aplicación está pensada para un usuario a la vez. Está preparada para no romper por un choque puntual (usa el modo WAL de SQLite y espera hasta 5 segundos si el fichero está ocupado), pero SQLite sobre una carpeta compartida de Windows es un problema conocido: el bloqueo de ficheros por red no es fiable y el riesgo real es corromper la base de datos. Si hacen falta dos personas a la vez, lo correcto es turnarse, o replantear el almacenamiento.

¿Envía algo a internet? No. No hay ni una sola llamada de red en el programa. Funciona igual con el equipo desconectado.

Se me ha borrado un cliente sin querer. No hay papelera: al borrar un cliente se borran en cascada sus paquetes de horas y todos sus servicios, y la ventana avisa de ello antes de hacerlo. La única vuelta atrás es la copia de seguridad. Por eso conviene pulsar **Copia de seguridad** de vez en cuando —y, en vez de borrar, desmarcar **Cliente activo**, que archiva sin perder nada.

8. Seguridad

Revisión completa del código fuente hecha el 4 de septiembre de 2026 sobre la versión 1.2.0: el fichero principal, las cinco ventanas, el acceso a datos, el generador de informes y los guiones de compilación.

Lo que se ha comprobado y estaba bien

- **Inyección SQL: no hay.** Todas las consultas de `Repo.cs` y `Db.cs` van parametrizadas (`AddWithValue`). Los dos únicos trozos de SQL que se montan concatenando texto (`WHERE c.activo = 1` y `AND c.activo = 1`) dependen de una casilla de la ventana, no de texto escrito por nadie. La búsqueda por texto también va parametrizada.
- **Inyección de HTML en los informes: no hay.** `Informe.cs` pasa por `WebUtility.HtmlEncode` el nombre del cliente, sus datos de contacto, los conceptos, las notas, las descripciones, los técnicos y los tipos. Un cliente llamado `<script>...` sale como texto, no como código.
- **Secretos, claves y volcados de base de datos: ninguno.** Se ha revisado la carpeta entera, incluidos los dos ficheros `.zip` de 60 y 75 MB. **No hay bases de datos con datos reales, ni copias de seguridad, ni credenciales, ni claves de API dentro del proyecto.** Lo que ocupa el espacio son los artefactos de compilación (`bin\`, `obj\`), el ejecutable publicado y los dos ZIP de respaldo del propio código. **No ha habido que mover nada a `utiles\`.**

- **Red:** cero peticiones. No hay HTTP, ni telemetría, ni comprobación de actualizaciones. El único acceso a internet de todo el repositorio está en `install-requisitos.ps1`, que descarga el instalador oficial de .NET desde `https://dot.net` (HTTPS) y solo lo ejecuta quien compile.
- **Ejecución de código:** no hay `eval`, ni deserialización de datos externos, ni carga dinámica de ensamblados.
- **Recorrido de rutas:** las rutas de guardado salen del diálogo estándar de Windows, y el nombre propuesto pasa por un saneador que sustituye los caracteres no válidos.

Lo que se ha arreglado en esta revisión

1. Inyección de fórmulas en el CSV exportado — `MainWindow.xaml.cs`, función `Csv()`. Era el fallo con consecuencias reales del lote. Excel y LibreOffice ejecutan como **fórmula** cualquier celda que empiece por `=`, `+`, `-` o `@`, **aunque venga entrecomillada**. Como la descripción del servicio y el nombre del técnico son texto libre, bastaba con anotar un servicio cuya descripción empezara por `=` para que, al abrir en Excel el CSV enviado al cliente, se ejecutara algo en su equipo. Ahora esos campos llevan delante un apóstrofo, que es justamente la marca de «esto es texto» de Excel y no se ve en la celda. Las duraciones que genera la propia aplicación (que pueden empezar por un signo menos cuando el saldo es negativo) se escapan por una vía aparte, `CsvDur()`, para que no les salga un apóstrofo delante sin motivo: no proceden de nada que haya escrito una persona.

2. Apertura de ficheros con extensión arbitraria — `MainWindow.xaml.cs`, función `Abrir()`. Después de exportar o de generar un informe, la aplicación abre el fichero con `UseShellExecute`, que **ejecuta** según la extensión. Si en el diálogo de guardar se escribía a mano un nombre acabado en `.exe`, `.bat`, `.cmd` o `.lnk`, abrirlo era lanzarlo. Ahora solo se abre lo que la aplicación genera (`.csv`, `.html`, `.htm`, `.txt`) y, además, los tres diálogos de guardar llevan `AddExtension` y `DefaultExt`, de modo que la extensión correcta se pone sola.

3. Ortografía castellana de toda la interfaz y de los informes — las cinco ventanas (`.xaml`), sus clases y `Informe.cs`. Lo que ve el cliente estaba escrito sin tildes ni ñes: «Duracion no valida», «DESCRIPCION», «TECNICO», «Anadir horas contratadas», «Se eliminaran las...», «pestanda», «Este ano», «Todo el historico». Se han corregido 61 caracteres en once ficheros, incluidas las cabeceras de las tablas de los dos informes imprimibles, que es precisamente el papel que acaba en manos del cliente. Los ficheros afectados se han guardado en UTF-8 **con marca de orden de bytes**, para que ni el compilador de C# ni el de XAML tengan que adivinar la codificación, y se ha verificado sobre el binario compilado que los textos acentuados están correctos y que no hay caracteres corruptos.

Hay **cuatro sitios que se han dejado sin tilde a propósito**, porque no son palabras sino contratos, y acentuarlos rompería el programa:

Fichero	Qué es
<code>MainWindow.xaml.cs</code> (cabecera del CSV)	Nombres de columna que lee Excel u otro proceso.
<code>Repo.cs</code> (descripción LIKE)	Nombre de columna de la base de datos.
<code>Informe.cs</code> ({E(titulo)})	Variable de C#.
<code>MainWindow.xaml</code> (Tag="Clasico")	Valor que se convierte con <code>Enum.TryParse</code> .

Quien vuelva a pasar el corrector automático sobre este proyecto debe excluirlos: aplicado a ciegas, los cambia y deja el programa sin compilar o con una consulta rota.

4. Número de versión — `ControlHoras.csproj` . El ejecutable se declaraba como 1.0.0 cuando la documentación decía 1.2.0. Ahora dice 1.2.0 en los tres campos de versión, que es lo que se ve en las propiedades del fichero en Windows.

Tras los cambios se ha recompilado el proyecto: **0 avisos y 0 errores**, y se ha regenerado `dist\ControlHoras.exe` .

Lo que queda pendiente, y por qué

Esto **no son fallos que se hayan pasado por alto**: son decisiones de diseño o cambios de comportamiento que no corresponde tomar en una revisión de seguridad. Se dejan escritas para que el cliente decida.

No hay contraseña, ni usuarios, ni permisos. Y no los hay porque no los ha habido nunca: no existe pantalla de acceso, ni contraseñas guardadas, ni sesión, ni distinción entre empleado y administrador. Quien tenga abierta la sesión de Windows del equipo ve y modifica todo. Para una herramienta de escritorio de un solo técnico es una decisión razonable —y tiene la virtud de que **no hay ninguna contraseña que se pueda guardar mal**—, pero hay que saberlo: **la protección real de estos datos es la contraseña de Windows del equipo**. Si el programa tuviera que compartirse entre varias personas con permisos distintos, eso es una funcionalidad nueva, no un ajuste.

La base de datos no está cifrada. Es un fichero SQLite corriente: cualquiera que pueda leerlo lo abre con un visor gratuito. En la ubicación por defecto (`%APPDATA%`) está dentro del perfil del usuario y protegido por los permisos de Windows, que es lo correcto. En **modo portable** —un pendrive o una carpeta de red— esa protección **desaparece**: el fichero queda al alcance de cualquiera con acceso a la carpeta. Como contiene nombres, CIF, personas de contacto, correos, teléfonos y el detalle de los trabajos, es un fichero con datos personales a efectos del RGPD. Recomendación: si va a un pendrive, cifrar la unidad (BitLocker); si va a una carpeta de red, restringir los permisos de esa carpeta a quien deba verlo. Lo mismo vale para la subcarpeta `backups\` , que hereda la ubicación de la base de datos.

Los paquetes de horas caducados siguen contando. `Bolsa.Caducada` está calculado en el modelo pero no se usa en ningún sitio: el saldo suma todos los paquetes, caducados o no. No se ha tocado porque cambiarlo **alteraría el saldo de clientes ya existentes**, que es exactamente lo que una revisión de seguridad no debe hacer por su cuenta. Si se quiere que las horas caduquen de verdad, es un cambio a decidir y a probar aparte.

Dos columnas muertas en la base de datos. `clientes.tarifa_hora` y `bolsas.precio_hora` existen en el esquema pero no las escribe ni las lee nadie: son restos de una idea anterior con precios. No estorban y borrarlas obligaría a migrar la tabla; se dejan documentadas.

El ejecutable no está firmado. De ahí el aviso de SmartScreen. Firmarlo exige comprar un certificado de firma de código.

El manejador global de errores traga todas las excepciones. `App.xaml.cs` muestra el mensaje y marca el error como tratado, de modo que la aplicación nunca se cierra sola. Es cómodo para el usuario, pero puede dejar la ventana mostrando datos de antes de un fallo; el remedio es volver a seleccionar el cliente, que recarga todo desde la base de datos.

9. Ficha técnica

Arquitectura. .NET 8 + WPF, publicación `SelfContained` + `PublishSingleFile` para `win-x64`, con compresión. Sin `Debug` (`DebugType=none`), lo que significa que tampoco se distribuyen los símbolos. Una única dependencia NuGet: `Microsoft.Data.Sqlite` 8.0.10.

Ficheros del proyecto (`src\ControlHoras\`):

Fichero	Qué contiene
<code>App.xaml</code> / <code>App.xaml.cs</code>	Arranque, cultura <code>es-ES</code> forzada, manejador global de errores y los tres convertidores de la interfaz.
<code>Db.cs</code>	Resolución de la ruta de la base de datos, apertura de conexiones, creación del esquema, tabla de ajustes y copia de seguridad.
<code>Repo.cs</code>	Todo el acceso a datos. Consultas parametrizadas.
<code>Models.cs</code>	<code>Cliente</code> , <code>Bolsa</code> , <code>Servicio</code> , los tipos de servicio y el cálculo del saldo y el estado.
<code>Duracion.cs</code>	Interpretación y formato de duraciones. Es el corazón del programa.
<code>Informe.cs</code>	Generación de los dos informes HTML.
<code>Tema.cs</code>	Cambio de aspecto en caliente y su persistencia.
<code>MainWindow.*</code>	Ventana principal: lista, saldo, barra de alta, pestañas, filtros, exportación e informes.
<code>ClienteDialog.*</code> , <code>BolsaDialog.*</code> , <code>ServicioDialog.*</code> , <code>PeriodoDialog.*</code>	Las cuatro ventanas auxiliares.
<code>Temas\Base.xaml</code> + <code>Clasico</code> , <code>Windows11</code> , <code>Mac</code> , <code>Crystal</code>	Las cuatro paletas.

Base de datos. SQLite. `PRAGMA journal_mode=WAL , foreign_keys=ON , busy_timeout=5000 .`

Tabla	Contenido	Claves e índices
clientes	id, nombre, cif, contacto, email, telefono, tarifa_hora (sin uso), notas, activo, creado	id autoincremental
bolsas	id, cliente_id, concepto, horas (decimal), precio_hora (sin uso), fecha_alta, fecha_caducidad, notas	FK a clientes con ON DELETE CASCADE ; índice por cliente_id
servicios	id, cliente_id, fecha, tipo, descripcion, tecnico, minutos (entero), facturable, creado	FK a clientes con ON DELETE CASCADE ; índices por cliente_id y por fecha
ajustes	clave, valor	Claves en uso: tema y tecnico_defecto

Las fechas se guardan como texto `AAAA-MM-DD` en cultura invariante, para que ordenen bien alfabéticamente. Las horas contratadas se guardan como decimales en `bolsas.horas` por compatibilidad, pero **se convierten a minutos enteros ya en el propio SQL** y todo el cálculo posterior va en enteros.

Fórmula del saldo:

```

contratadas = ROUND(SUM(bolsas.horas) * 60)
consumidas  = SUM(servicios.minutos) donde facturable = 1
restantes   = contratadas - consumidas           (puede ser negativo)
porcentaje  = min(100, consumidas * 100 / contratadas)

estado = SINBOLSA si contratadas <= 0
        AGOTADO  si restantes  <= 0
        AVISO    si porcentaje >= 80
        OK       en el resto

```

Rutas en el equipo:

Qué	Dónde
Base de datos (normal)	<code>%APPDATA%\ControlHoras\controlhoras.db</code>
Base de datos (portable)	<code>controlhoras.db</code> o <code>datos\controlhoras.db</code> junto al <code>.exe</code>
Copias de seguridad	<code>backups\controlhoras_AAAAMDD_HHMMSS.db</code> junto a la base de datos
Informes y CSV	Donde los guarde el usuario en el diálogo

No hay ficheros de registro, ni ficheros de configuración, ni tareas programadas, ni servicios, ni entradas en el registro de Windows, ni carpetas temporales propias.

Compilar:

```
powershell -ExecutionPolicy Bypass -File .\install-requisitos.ps1 # solo la primera vez
powershell -ExecutionPolicy Bypass -File .\build.ps1 # -> dist\ControlHoras.exe
powershell -ExecutionPolicy Bypass -File .\build.ps1 -Run # compilar y abrir
powershell -ExecutionPolicy Bypass -File .\tools\generar-icone.ps1 # regenerar el icono
```

Control de Horas 1.2.0 — Ecom Experts — Licencia AFL-3.0